第 5 章練習詳解：行程同步

OS05-E01　臨界區的三項性質

1. 題目中的無限次插隊違反有限等待（bounded waiting）：一旦某執行緒請求進入，其他參與者在它進入前的次數必須有界。互斥則只要求任何時刻至多一人位於臨界區；一個演算法可以滿足互斥，卻使特定等待者飢餓。
2. 不必然。若其他執行緒一直進出臨界區，系統仍可能有整體進展，但被永久跳過的執行緒發生 starvation。Deadlock 則指等待關係使相關工作都無法前進；兩者不能互相等同。題目只保證互斥和被無限超越，不能單靠它判定 progress 必然成立。
3. 從 Pi 成功原子取票起算，固定 n 人且每人最多一個未完成請求時，Pi 前面最多有 n−1 張尚未完成的票。若票號不繞回、依票序服務，後來的請求不能超越 Pi；在每位前任最終離開、Pi 最終獲排程的假設下，Pi 最多在 n−1 次其他參與者進入後進入。這是進入次數的上界；除非再給執行時間與排程延遲上界，不能直接推出固定時間上界。

OS05-E02　有界緩衝區的鎖定順序

1. Producer 先 `wait(empty)`，成功保留一個空槽後才 `wait(mutex)`。目前 `empty=0`，所以它在持有 mutex 之前等待；既有三項仍可由 consumer 取出。
2. Consumer 先成功 `wait(full)`、取得 mutex 並取出一項，再依序 `signal(mutex)`、`signal(empty)` 是一種正確做法。先完成資料結構更新，再發布空槽；被喚醒的 producer 仍須取得 mutex 才能改動緩衝區。
3. 錯誤順序下，producer 持有 mutex 且等待 empty；consumer 雖可取得 full permit，卻在等待 producer 持有的 mutex。只有 consumer 取得 mutex 並完成取出後才會通知 empty，形成 producer → empty → consumer → mutex → producer 的等待環；兩者都不能完成。

OS05-E03　條件變數與狀態條件

1. Condition variable 不保存一個可供未來等待者消費的 signal。在 Mesa 語意下，等待者被喚醒後仍須重取鎖；期間其他執行緒可能先消耗資料或改變條件。虛假喚醒或 broadcast 也使「收到通知」不等於 `ready` 必為 true。
2. 等待者執行 `lock(m); while (!ready) wait(cv,m); use(buffer); unlock(m)`；通知者在同一 `m` 的保護下更新 `ready` 及資料、通知，並按所用 API 的契約釋鎖。`wait` 在持鎖呼叫，將釋鎖與提交等待同通知側形成原子交接；每次返回後已重新取得 `m`，仍須重查條件。
3. 若通知先發生而 `ready` 在檢查時仍為 true，等待者直接通過 while，無須等待先前 signal。若在它取得鎖之前其他人已將 `ready` 改回 false，它應等待未來使條件再次成立的事件；這是條件目前不成立，而不是遺失喚醒。若資料仍可用但等待者因檢查到登記之間的競賽錯過唯一通知，才是本題所防止的 lost wakeup。

OS05-E04　Peterson 演算法與記憶體順序

1. 反設 P0、P1 同時在各自本輪臨界區。比較兩人本輪進入前寫入 `turn` 的先後。若 P1 的 `turn=0` 較晚，P0 在本輪的 `flag[0]=true` 先於其 `turn=1`，也先於 P1 的 `turn=0`；P0 尚未離開，未清除旗標。P1 在該次寫入 turn 後檢查時，P0 本輪不會再改寫 turn，因此在 SC 次序下可見 `flag[0]=true` 且 `turn=0`，P1 的等待條件成立，不能進入，矛盾。若 P0 的 `turn=1` 較晚，對稱地使 P0 受阻。故兩個臨界區不能重疊。
2. 從 Pi 完成 `flag[i]=true; turn=j` 起算，Pj 若已在臨界區或已通過進入條件，可能先進入一次。Pj 離開並重新請求時必須先寫 `turn=i`；只要 Pi 的旗標仍為 true，Pj 就不能在 Pi 之前再次通過自己的等待條件，故最多一次。若只從 Pi 設旗標起算，Pj 可能早已通過檢查、先進入一次，重新請求並設 `turn=i` 後等待；Pi 隨後才設 `turn=j`，Pj 可再進入一次。起算點不能省略。
3. 單方請求時，若對方旗標為 false，請求者可通過；雙方同時競爭時，turn 決定至少一方不受等待條件阻擋，因此在題設公平排程與臨界區最終結束等假設下滿足 progress。第 1 小題證互斥，第 2 小題給 bounded waiting。有限等待限制其他人進入的次數，不自動給出以秒計的時間上界；若執行緒一直未被排程，實際延遲仍可能無界。
4. 一般非原子 `flag`、`turn` 的跨執行緒衝突讀寫缺乏 happens-before 時，在 C/C++ 中構成 data race、行為未定義；編譯器與 CPU 也不必呈現 SC 的單一全域次序。原子操作必須採足夠的記憶體順序，或使用經證明的同步協定，不能只是把非原子程式當作 SC 交錯執行。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

OS05-E05　Semaphore 與緩衝區狀態

1. 下表均為每項操作完整結束、沒有在途 reservation 的邊界；mutex 因臨界區已結束而回到 1。

完成的操作	empty	full	mutex	FIFO 內容
初始	3	0	1	[]
放 A	2	1	1	[A]
放 B	1	2	1	[A,B]
放 C	0	3	1	[A,B,C]
取 A	1	2	1	[B,C]
放 D	0	3	1	[B,C,D]

2. 抽象 binary semaphore 有一個許可：第一個成功 wait 後沒有可用許可，第二個 wait 必須等到第一個持有者 signal 才能通過。以題目指定的一種實作說明：計數 S 初值 1；T0 的 wait 令 S：1→0 並持有許可；T1 的 wait 令 S：0→−1 並入等待佇列。T0 signal 令 S：−1→0，將許可原子交給一名等待者；T1 被喚醒後延續原本的 wait，不再重複扣減。T1 隨後 signal 令 S：0→1，留下可用許可。負數、佇列及交棒是這個模型的實作方式，不能當成所有 semaphore 的原始計數值或公平保證。

3. Producer 持有 mutex，等待空槽；consumer 即使取得一個 full permit，也在等待 mutex，無法取出項目並通知 empty。等待關係為 producer → empty → consumer → mutex → producer；若沒有其他介入，兩者 deadlock。

4. 設 E、F 為尚未保留的 empty／full 許可數，P 為 producer 已取得空槽、尚未發布為 full 的數目，C 為 consumer 已取得 full、尚未發布為 empty 的數目。在此容量模型下 E+F+P+C=3；例如一個 producer 剛成功 wait(empty) 後為 (E,F,P,C)=(2,0,1,0)，故 E+F=2。直接交棒的許可也算已授出的 reservation。若某 semaphore 實作的有號原始值包含等待者，它不能直接當成非負的 E 或 F；此外仍須保持實際項目數在 0 至 3，且先完成資料更新再發布相應許可。

OS05-E06　Mesa、Hoare 與遺失喚醒

1. 若等待者先解鎖、稍後才登記，通知者可在兩步之間持鎖使條件成立並 signal，但此時沒有已登記的等待者；等待者隨後睡下，即使條件已真也可能無人再喚醒。正確 wait 對通知側提供釋鎖與提交等待的原子交接，防止這個空隙。
2. 通知先到且 `ready` 到檢查時仍為 true，等待者持鎖檢查後不呼叫 wait。若條件已被其他受鎖保護的操作改回 false，等待者重回 wait 是正確地等待下一次條件成立，並非 signal 因競賽遺失；它能否最終完成還取決於未來是否有事件使條件成立及排程公平性。
3. Mesa 的 signal 不立即把 monitor 交給等待者；等待者重取鎖前，其他執行緒可改變條件。因此它看到 false 時必須繼續 while 並再次 wait，不能直接使用 buffer。
4. 在題設 Hoare 模型中，signaler 先把 predicate 建立為 true，signal 立即將 monitor 交給等待者；在受保護狀態未被其他人介入、沒有虛假喚醒或逾時時，等待者可依該交接前提繼續。立即交接可能帶來額外排程／上下文切換，signaler 也必須等待。Mesa 讓 signaler 繼續持有 monitor，等待者之後重新競爭鎖，因此須用 while 重查。立即交接本身不會替程式建立原本為 false 的 predicate；兩種模型都要求狀態更新與通知協定正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

OS05-E07　被通知者沒有拿到唯一項目

1. 以下每列是事件完成後的狀態；E 表示已被通知但尚未重新取得 m 的執行緒。

事件	m 持有者	q	CV 等待者	E
T0：C1 wait	無	[]	{C1}	{}
T1：P 放 A	P	[A]	{C1}	{}
T2：P signal C1	P	[A]	{}	{C1}
T3：P 釋鎖，C2 取 A 後釋鎖	無	[]	{}	{C1}
T4：C1 重取鎖	C1	[]	{}	{}
T5：C1 重新 wait	無	[]	{C1}	{}

2. 用 if 時，C1 從 wait 返回後不重查，第一個錯誤是試圖對空 q 執行 pop。通知已送到 C1，且題設沒有虛假喚醒；真正原因是 C2 在 C1 重取鎖前合法地取走了唯一項目。
3. 錯誤拆分的交錯：① C1 持鎖發現 q 空後先 unlock，尚未登記；② P 取得 m、放入 A、signal（此時 CV 無等待者）並 unlock；③ C1 才登記並阻塞。此時 q=[A]，C1 卻可能無人再喚醒。這不是合法的 `pthread_cond_wait` 用法：呼叫時需持有 mutex，且 wait 提供相對通知側的原子交接。
4. Broadcast 使兩個等待者可競爭重新取得鎖，並不複製 A。若沒有其他 producer，先取得鎖且看到 A 的 consumer 可取走它；另一人重查時發現 q 空，便繼續等待。While 在正確鎖協定下保護「空佇列不 pop」的安全性，並不保證 FIFO、公平或未來一定有新資料。

OS05-E08　Bakery 選號與同號

1. 對應的配對為 (1,0)、(1,1)，按字典順序由 P0 優先。讀取最大值的過程不是一個不可分割的快照，兩者都可能在對方發布前看到 0。
2. P0 算出 1 尚未發布；P1 算出並發布 1，看到 number[0]＝0 後進入。P0 再發布 1，看到自己的 (1,0) 優先於 (1,1)，也進入。正確版本中 P1 會先等 `choosing[0]` 變成 false。
3. Pi 完成取號時，其他參與者至多有兩個現存或正在選號的請求；這些請求可能排在它之前。其他參與者完成後若再次請求，當 Pi 的號碼仍有效，新票號會排在 Pi 之後。最終進入還須選號和臨界區完成、相關執行緒持續獲排程；次數界不是等待時間界。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 5. Lamport, A New Solution of Dijkstra’ s Concurrent Programming Problem, 1974. pthread_cond_wait(3p); Windows spin locks; Windows critical sections; Linux lock types; OpenMP critical.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study